

Mini-Projet B - Rapport

I. Calculs des performances

A l'aide de Pandas, nous affichons dans un tableau comparatif l'erreur et le temps d'exécution de nos trois méthodes pour différents nombre de segments lors du calcul de l'intégrale de la fonction polynomiale $f(x)=1+x+x^2+x^3$.

Concernant l'erreur absolue, nous pouvons d'abord préciser que le codage n'impacte pas sa valeur (python de base versus Numpy) puisque les deux codages retournent la même solution. Nous remarquons que la méthode des rectangles provoque une erreur absolue très importante. La méthode des trapèzes présente une erreur bien plus faible mais tout de même notable, tandis que la méthode de Simpson n'a quasiment pas d'erreur (ordre 10^{-10}). Cela s'explique par l'arrondi numérique proche de zéro puisque la méthode de Simpson est théoriquement exacte pour une fonction polynomiale de degré 3 que nous étudions ici. L'erreur absolue des méthodes des rectangles et trapèzes se réduit proportionnellement par rapport au nombre de segments (rapport 1 et 10 respectivement).

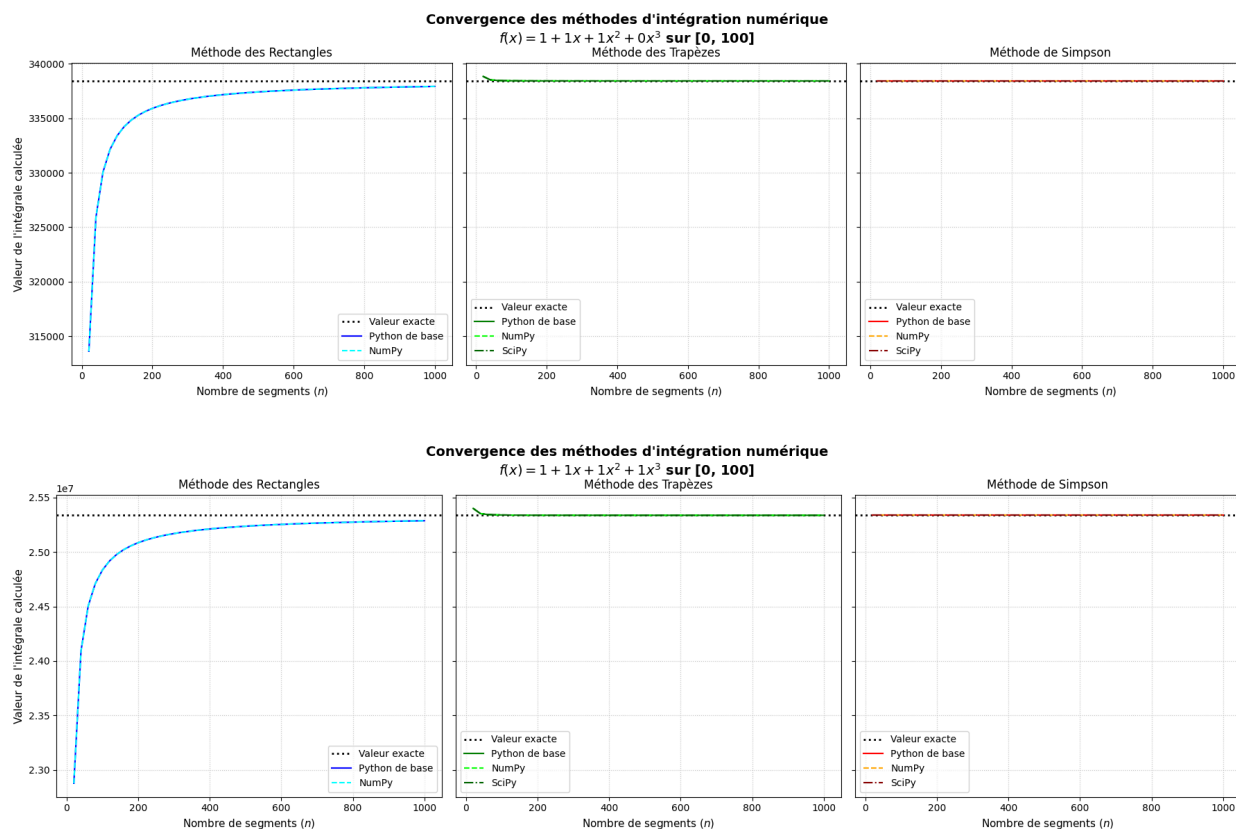
Concernant le temps d'exécution, l'apport de Numpy est flagrant : la vectorisation permet de réduire le temps d'exécution par environ 3,5 pour 100 segments, 14 pour 1000 segments et 24 pour 10000 segments par rapport au codage en Python de base. On remarque que la méthode des trapèzes permet un gain encore plus important que pour les deux autres méthodes: facteur 26.2 pour 1000 et 47.6 pour 10000 segments. Nous pouvons également comparer les méthodes entre elles: le temps d'exécution évolue de façon cohérente avec la complexité des méthodes: la méthode des rectangles est la plus rapide, suivie de la méthode des trapèzes puis celle de Simpson.

TABLEAU COMPARATIF MULTI-ÉCHELLES DE SEGMENTS (n)										
Segments		n = 100			n = 1000			n = 10000		
Version		Python de base	Numpy	Gain	Python de base	Numpy	Gain	Python de base	Numpy	Gain
Métrique	Méthode									
Erreur Absolue	Rectangles	502533.3333333321	502533.3333333321	-	50479.8333333060	50479.8333333284	-	5050.2403333088	5050.2483333312	-
	Trapèzes	2516.6666666679	2516.6666666679	-	25.1666666754	25.1666666716	-	0.2516666651	0.2516666688	-
	Simpson	0.0000000037	0.0000000037	-	0.0000014156	0.0000000000	-	0.0000144355	0.0000000037	-
Temps total (s)	Rectangles	0.0057	0.0017	3.4x	0.0580	0.0042	13.9x	0.5638	0.0234	24.1x
	Trapèzes	0.0113	0.0022	5.1x	0.1186	0.0045	26.2x	1.1355	0.0239	47.6x
	Simpson	0.0161	0.0045	3.6x	0.1617	0.0112	14.5x	1.6066	0.0681	23.6x

II. Vérification de la convergence

L'analyse des courbes de convergence pour les fonctions $f(x) = 1+x+x^2$ et $f(x)=1+x+x^2+x^3$ met en évidence l'efficacité relative des méthodes d'intégration numérique que nous avons codées. On observe une hiérarchie claire dans la vitesse d'approximation : la méthode de Simpson atteint la valeur théorique exacte de manière quasi instantanée, suivie par la méthode des trapèzes qui se stabilise rapidement (<100 segments). À l'inverse, la méthode des rectangles affiche la convergence la plus lente, n'atteignant pas la solution exacte même pour 1000 segments.

Par ailleurs, les résultats en Python de base, NumPy et SciPy se superposent strictement pour chaque algorithme. Cela démontre que la vectorisation n'influence pas la justesse numérique du calcul mais seulement le temps d'exécution, et que notre codage permet d'obtenir le même résultat que les fonctions préprogrammées de la bibliothèque SciPy. Enfin, la similitude des deux graphiques indique que cette vitesse de convergence est une propriété intrinsèque à chaque méthode, indépendante du polynôme étudié.

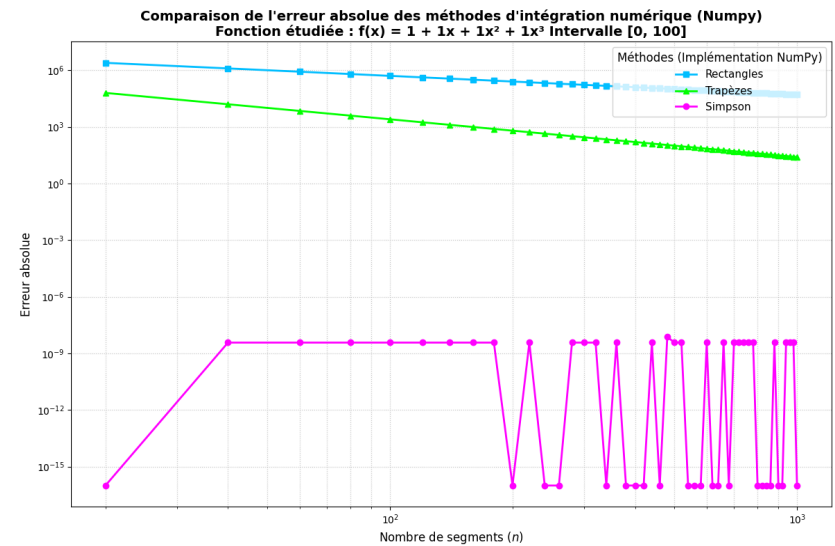
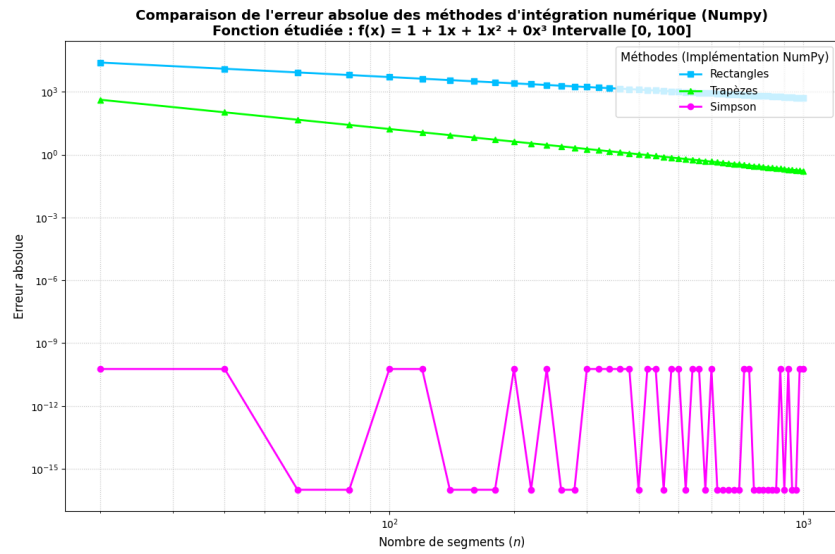
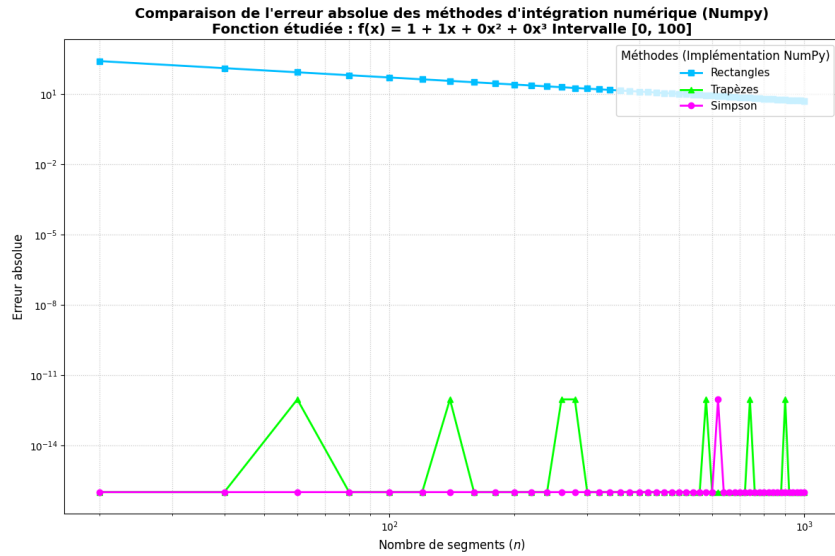


III. Comparaison de l'erreur absolue

Nous comparons l'évolution de l'erreur absolue en fonction du nombre de segments pour trois fonctions polynomiales de degrés croissants (ordres 1, 2 et 3). Les courbes sont tracées en échelle double-logarithmique. Ici nous avons tracé les résultats obtenus par le codage Numpy arbitrairement, les résultats étant égaux peu importe l'implémentation.

Globalement, on observe que la précision progresse logiquement avec l'augmentation de n et avec la complexité de la formulation géométrique.

En effet, la méthode des rectangles est la moins précise, suivie de la méthode des trapèzes (sauf pour ordre 1) puis de la méthode de Simpson dont l'erreur absolue est nulle. Les variations chaotiques visibles pour la méthode de Simpson et des trapèzes (ordre 1) sont dûes au bruit numérique causé par les erreurs d'arrondi de la machine, l'erreur théorique dans ces cas étant nulle.



IV. Comparaison du temps d'exécution

Nous avons ensuite cherché à comparer les performances temporelles de chaque codage: python de base, Numpy et la fonction de Scipy. Nous pouvons constater que pour les deux méthodes (Trapèzes et Simpson), et pour les 3 fonctions polynomiales étudiées (ordre 1,2 et 3), la fonction codée en Python est largement plus lente que les autres, suivie de celle en Numpy et enfin celle de Scipy. Le codage en Python de base augmente grandement avec le nombre de segments tandis que Numpy et Scipy sont moins impactés par ce facteur, et augmentent plus lentement. Ces graphiques nous indiquent que les fonctions incluses dans la bibliothèque Scipy sont bien plus performantes en termes de temps d'exécution. Cela s'explique par la gestion de la mémoire en C, sans passer par des tableaux intermédiaires.

